

Sentiora — API Specification

Version: 1.0 **Status:** Draft for Engineering Implementation **Base URL:** <https://api.sentiora.com/api/v1> **Audience:** Frontend engineers, Chrome Extension engineers, Backend engineers, QA **Companion documents:** PRD v2.1 (FINAL), Backend Technical Specification, Database Schema

This document defines the external API contract only. It does not restate product vision, architecture, database schema, or UI flows. Where those are relevant, they are covered in the companion documents referenced above.

Table of Contents

1. Conventions & Design Rules
 2. Authentication
 3. Standard Response Format
 4. HTTP Status Codes
 5. API Versioning
 6. Pagination, Sorting, Filtering, Searching
 7. Rate Limiting
 8. Idempotency
 9. File Uploads
 10. Module: Authentication
 11. Module: User Profile
 12. Module: Memory
 13. Module: Capture
 14. Module: Upload
 15. Module: Search
 16. Module: AI
 17. Module: Collections
 18. Module: Tags
 19. Module: Dashboard
 20. Module: Notifications
 21. Module: Chrome Extension
 22. Module: Admin (Future)
 23. Webhooks (Future)
 24. Security
 25. Error Code Catalog
 26. OpenAPI / Documentation Strategy
 27. API Naming Rules
-

1. Conventions & Design Rules

- All APIs are REST over HTTPS. No GraphQL, no gRPC.
- All request/response bodies are JSON (Content-Type: application/json), except file upload endpoints (multipart/form-data).
- All routes are prefixed with /api/v1/.
- Resource URLs use plural nouns, lowercase, kebab-case for multi-word resources (e.g. /saved-searches, not /savedSearches or /saved_search).
- Standard HTTP methods only: GET, POST, PUT, PATCH, DELETE.
- All timestamps are ISO 8601 UTC (2026-07-30T14:32:00Z).
- All IDs are UUIDv4 strings unless otherwise noted.
- Every request is assigned a request_id (server-generated, echoed in the response) for traceability.

2. Authentication

Sentiora uses **JWT-based authentication** with short-lived access tokens and long-lived refresh tokens.

Token	Lifetime	Storage (Web)	Storage (Extension)
Access Token	15 minutes	Memory / HttpOnly cookie (preferred)	chrome.storage.session
Refresh Token	30 days (sliding)	HttpOnly, Secure, SameSite=Strict cookie	chrome.storage.local (encrypted at rest by OS keychain where available)

2.1 Authorization Header

All protected endpoints require:

Authorization: Bearer <access_token>

2.2 Public vs Protected Endpoints

Public (no auth required): POST /auth/register, POST /auth/login, POST /auth/forgot-password, POST /auth/reset-password, POST /auth/refresh-token, GET /health, GET /version

Protected (JWT required): All other endpoints in this specification.

2.3 Token Expiration & 401 Handling

- If the access token is expired or invalid, the API returns 401 UNAUTHORIZED with error code AUTH_TOKEN_EXPIRED or AUTH_TOKEN_INVALID.
- Clients must attempt a silent refresh via POST /auth/refresh-token exactly once. If refresh fails, the client must force re-login and clear local tokens.
- The API never returns a WWW-Authenticate challenge; JWT state is managed entirely via response error codes.

2.4 403 Handling

- 403 FORBIDDEN is returned when the token is valid but the user lacks permission for the resource (e.g. accessing another user's memory, or an unverified account calling a verified-only endpoint). Error code: AUTH_FORBIDDEN.
- 403 responses never leak whether the resource exists — they are indistinguishable from “resource belongs to someone else.”

2.5 Refresh Flow

POST /api/v1/auth/refresh-token

Request Body:

```
{ "refresh_token": "eyJhbGciOi..." }
```

Success (200):

```
{  
  "success": true,  
}
```

```

      "data": {
        "access_token": "eyJhbGciOi...",
        "refresh_token": "eyJhbGciOi...",
        "access_token_expires_in": 900
      },
      "meta": { "timestamp": "2026-07-30T14:32:00Z", "request_id":
"req_9f3c" }
    }
  }

```

Refresh tokens are **rotated on every use** (old refresh token is invalidated immediately). Reuse of an already-rotated refresh token invalidates the entire token family and forces re-login (replay-attack defense — see Security §24).

2.6 Logout

POST /api/v1/auth/logout

Invalidates the current refresh token family server-side. Access tokens are not individually revocable (by design, given 15-minute TTL) but the client must discard them immediately.

3. Standard Response Format

3.1 Success Envelope

```

{
  "success": true,
  "data": { },
  "meta": {
    "timestamp": "2026-07-30T14:32:00Z",
    "request_id": "req_9f3c1a2b"
  }
}

```

3.2 Paginated Success Envelope

```

{
  "success": true,
  "data": [ ],
  "pagination": {
    "cursor": "eyJpZCI6IjEyMyJ9",
    "next_cursor": "eyJpZCI6IjQ1NiJ9",
    "has_more": true,
    "page_size": 20
  },
  "meta": {
    "timestamp": "2026-07-30T14:32:00Z",
    "request_id": "req_9f3c1a2b"
  }
}

```

3.3 Failure Envelope

```

{
  "success": false,
  "error": {
    "code": "MEMORY_NOT_FOUND",
    "message": "The requested memory does not exist or has been
deleted.",
    "details": null
  },
}

```

```

    "meta": {
      "timestamp": "2026-07-30T14:32:00Z",
      "request_id": "req_9f3c1a2b"
    }
  }
}

```

3.4 Validation Error Envelope

```

{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more fields failed validation.",
    "details": [
      { "field": "email", "issue": "Must be a valid email address." },
      { "field": "password", "issue": "Must be at least 10
characters." }
    ],
  },
  "meta": {
    "timestamp": "2026-07-30T14:32:00Z",
    "request_id": "req_9f3c1a2b"
  }
}

```

Validation errors always use HTTP 422 and error code `VALIDATION_ERROR`, with a details array of field-level issues.

4. HTTP Status Codes

Code	Usage
200 OK	Successful GET/PUT/PATCH/DELETE with response body
201 Created	Resource successfully created (POST)
202 Accepted	Request accepted for async processing (e.g. capture job queued)
204 No Content	Successful action with no response body (e.g. delete confirmation in some cases)
400 Bad Request	Malformed request (bad JSON, missing required header)
401 Unauthorized	Missing, invalid, or expired token
403 Forbidden	Authenticated but not permitted
404 Not Found	Resource does not exist or not visible to caller
409 Conflict	State conflict (e.g. duplicate email on register, duplicate tag name)
413 Payload Too Large	Upload exceeds max size
415 Unsupported Media Type	Disallowed MIME type on upload
422 Unprocessable Entity	Validation failure on otherwise well-formed request
429 Too Many Requests	Rate limit exceeded
500 Internal Server Error	Unhandled server-side failure

5. API Versioning

- Current version: `/api/v1/`.
- Version is expressed only in the URL path (not headers), for simplicity across the Chrome Extension, web, and third-party integrations.
- Breaking changes require a new version prefix (`/api/v2/`). Non-breaking additive changes (new optional fields, new endpoints) ship within `v1`.
- Deprecated endpoints are marked with a Deprecation response header containing a sunset date (Deprecation: true, Sunset: 2027-01-01T00:00:00Z) for a minimum of 90 days before removal.
- Deprecated endpoints continue to function unchanged during the deprecation window; no silent behavior changes are permitted during that period.

6. Pagination, Sorting, Filtering, Searching

6.1 Pagination Strategy: Cursor-based (recommended)

Cursor-based pagination is used for all list endpoints instead of offset-based, because:

- Memory, Capture, and Notification lists are high-write, frequently-inserted collections — offset pagination would produce duplicate/skipped rows as new items are captured in real time.
- Cursor pagination performs consistently at scale with pgvector-backed tables, avoiding `OFFSET` cost on large tables.

Query parameters (list endpoints):

Param	Type	Default	Notes
cursor	string	none	Opaque, base64-encoded. Omit for first page.
limit	integer	20	Max 100
sort	string	- created_at	Prefix - for descending
filter[field]	string	—	e.g. filter[source_type]=youtube
q	string	—	Free-text search within the resource (where supported)

6.2 Limits

- Default page size: **20**
- Maximum page size: **100**
- Requests for `limit > 100` return 422 `VALIDATION_ERROR`.

7. Rate Limiting

Limits are enforced per-user (authenticated) or per-IP (public/auth endpoints), using a sliding window.

Category	Limit	Window
Auth (login, register, forgot-password)	10 requests	15 minutes
Capture APIs	60 requests	1 minute
Search APIs (semantic, keyword, hybrid)	120 requests	1 minute
AI APIs (ask, summarize, explain)	30 requests	1 minute
Upload APIs	20 requests	5 minutes
All other authenticated APIs	300 requests	1 minute

429 Response:

```
{
  "success": false,
  "error": {
    "code": "SYSTEM_RATE_LIMITED",
    "message": "Rate limit exceeded. Please retry later.",
    "details": { "limit": 30, "window_seconds": 60 }
  },
  "meta": { "timestamp": "2026-07-30T14:32:00Z", "request_id":
"req_9f3c" }
}
```

Response includes header: Retry-After: 42 (seconds).

8. Idempotency

The following endpoints **require** an Idempotency-Key header (client-generated UUIDv4) to safely handle retries:

- POST /memories (manual create)
- POST /captures/webpage, POST /captures/youtube, POST /captures/article, POST /captures/pdf, POST /captures/bulk
- POST /uploads/document, POST /uploads/pdf, POST /uploads/image
- POST /ai/ask

If a request is received with a previously-used Idempotency-Key for the same user, the API returns the original response (HTTP 200/201, cached for 24 hours) rather than creating a duplicate resource. Reuse of the key with a **different** request body returns 409 CONFLICT (SYSTEM_IDEMPOTENCY_KEY_REUSED).

9. File Uploads

- All uploads use multipart/form-data with a single file field plus optional metadata fields.
- **Allowed MIME types:** application/pdf, image/png, image/jpeg, image/webp, text/plain, text/markdown.
- **Maximum size:** 25 MB per file (PDF/image), 2 MB for plain text.
- Files are scanned by an async malware/security validation step (ClamAV or equivalent) before being marked ready; until then the upload status is processing.
- Passwords, banking, and health-related documents are **never** accepted for capture — see Never-Capture Policy referenced in the Backend Technical Specification. This API layer enforces client-declared content_category rejection but does not perform content classification (that is a backend/AI concern, out of scope here).

Failure responses: - 413 PAYLOAD_TOO_LARGE → UPLOAD_FILE_TOO_LARGE - 415 UNSUPPORTED_MEDIA_TYPE → UPLOAD_INVALID_MIME_TYPE - 422 → UPLOAD_VALIDATION_FAILED (e.g. corrupt PDF, zero-byte file)

10. Module: Authentication

10.1 Register

POST /auth/register — Public

Body:

```
{ "email": "vansh@example.com", "password": "StrongPass123!",  
  "full_name": "Vansh Kumar" }
```

Validation: email format; password ≥ 10 chars, 1 upper, 1 number; full_name 2-80 chars.

201 Response:

```
{ "success": true, "data": { "user_id": "usr_1a2b", "email":  
  "vansh@example.com", "email_verified": false } }
```

Errors: 409 AUTH_EMAIL_ALREADY_EXISTS, 422 VALIDATION_ERROR

10.2 Login

POST /auth/login — Public **Body:** { "email": "...", "password": "..."} **200 Response:** access_token, refresh_token, user summary object.
Errors: 401 AUTH_INVALID_CREDENTIALS, 429 SYSTEM_RATE_LIMITED

10.3 Logout

POST /auth/logout — Auth required. No body. **204 No Content.**

10.4 Refresh Token

POST /auth/refresh-token — Public (token itself is the credential). See §2.5.

10.5 Forgot Password

POST /auth/forgot-password — Public **Body:** { "email": "..."} **202 Response:** Always returns success regardless of whether the email exists (prevents user enumeration).

10.6 Reset Password

POST /auth/reset-password — Public **Body:** { "reset_token": "...", "new_password": "..."} **200 / Errors:** 400 AUTH_RESET_TOKEN_INVALID, 422 VALIDATION_ERROR

10.7 Change Password

PUT /auth/change-password — Auth required **Body:** { "current_password": "...", "new_password": "..."} **Errors:** 401 AUTH_INVALID_CREDENTIALS (wrong current password)

10.8 Current User

GET /auth/me — Auth required. Returns the authenticated user’s core identity object.

10.9 Delete Account (Future Placeholder)

DELETE /auth/account — **Not implemented in MVP.** Reserved for V1.x. Will require re-authentication + confirmation flow + async data purge job.

11. Module: User Profile

Method	Endpoint	Purpose
GET	/users/me/profile	Fetch profile (name, avatar, bio, timezone)
PATCH	/users/me/profile	Update profile fields
GET	/users/me/preferences	General app preferences
PATCH	/users/me/preferences	Update preferences
GET	/users/me/ai-settings	AI response tone/verbosity, model preference
PATCH	/users/me/ai-settings	Update AI settings
GET	/users/me/privacy-settings	Per-source capture toggles, never-capture confirmations
PATCH	/users/me/privacy-settings	Update privacy/consent settings
GET	/users/me/notification-settings	Email/push notification preferences
PATCH	/users/me/notification-settings	Update notification settings

Example — Update Privacy Settings PATCH
/api/v1/users/me/privacy-settings

```
{
  "capture_sources": {
    "webpage": true,
    "youtube": true,
    "pdf": true,
    "github": false,
    "ai_conversations": true
  },
  "never_capture_acknowledged": true
}
```

Validation: capture_sources keys must be a subset of the supported source enum. Unknown keys → 422 VALIDATION_ERROR.

12. Module: Memory

Base resource: /memories

Method	Endpoint	Purpose
POST	/memories	Create memory (manual entry)
GET	/memories/{memory_id}	Get single memory
PUT	/memories/{memory_id}	Full update
PATCH	/memories/{memory_id}	Partial update
DELETE	/memories/{memory_id}	Soft-delete memory
POST	/memories/{memory_id}/archive	Archive
POST	/memories/{memory_id}/restore	Restore from archive/trash
GET	/memories	List memories (paginated)
GET	/memories?filter[...]	Filter memories (source_type, date_range, tag)
GET	/memories/search-metadata	Search by title/metadata only (non-semantic)
GET	/memories/recent	Recently captured (last 7 days, capped)
GET	/memories/pinned	Pinned memories
GET	/memories/favorites	Favorited memories
POST	/memories/{memory_id}/pin	Pin
DELETE	/memories/{memory_id}/pin	Unpin
POST	/memories/{memory_id}/favorite	Favorite
DELETE	/memories/{memory_id}/favorite	Unfavorite

12.1 Create Memory

POST /api/v1/memories **Auth:** required **Body:**

```
{
  "title": "Understanding pgvector indexing",
  "content": "Notes on HNSW vs IVFFlat...",
  "source_type": "manual",
  "source_url": null,
  "tags": ["postgres", "vector-search"]
}
```

Validation: title 1-200 chars; content required, $\leq 50,000$ chars; source_type $\in$ {manual, webpage, youtube, pdf, article, github, ai_conversation}.

201 Response:

```
{ "success": true, "data": { "memory_id": "mem_7f2a", "status": "ready", "created_at": "2026-07-30T14:32:00Z" } }
```

12.2 Get Memory

GET /memories/{memory_id} $\rightarrow$ 404 MEMORY_NOT_FOUND if absent/not owned.

12.3 Delete Memory

DELETE /memories/{memory_id} — soft delete (moves to trash, restorable for 30 days). **204 No Content.**

12.4 List / Filter Memories

GET /memories?filter[source_type]=youtube&filter[date_from]=2026-06-01&filter[date_to]=2026-07-01&sort=-created_at&limit=20&cursor=...

Notes: Filtering, sorting, and pagination follow the conventions in §6. Combining q with filter[...] performs metadata-only matching — for meaning-based results, use the Search module (§15).

13. Module: Capture

Base resource: /captures

Method	Endpoint	Purpose
POST	/captures/manual	Manual save (URL or text snippet)
POST	/captures/webpage	Capture a webpage (Meaningful Capture Engine applied)
POST	/captures/youtube	Capture a YouTube video (metadata + transcript)
POST	/captures/article	Capture an article page
POST	/captures/pdf	Capture an already-uploaded PDF (references upload_id)
POST	/captures/bulk	Submit multiple capture jobs in one call
GET	/captures/{capture_id}/status	Poll capture job status
POST	/captures/{capture_id}/retry	Retry a failed capture
POST	/captures/{capture_id}/cancel	Cancel a queued/in-progress capture

13.1 Capture Webpage

POST /api/v1/captures/webpage **Headers:** Idempotency-Key: <uuid>

Body:

```
{
  "url": "https://example.com/article",
  "page_title": "Example Article",
  "dwell_time_seconds": 145,
  "engagement_signal": "high",
  "html_snapshot_ref": null
}
```

Validation: url must be valid https URL; dwell_time_seconds ≥ 0 ; rejected if URL matches a known sensitive-page pattern (login, banking, health portal) — see Never-Capture Policy.

202 Response (async job accepted):

```
{
  "success": true,
  "data": { "capture_id": "cap_44de", "status": "queued",
"estimated_completion_seconds": 8 }
}
```

Errors: 422 CAPTURE_URL_BLOCKED_SENSITIVE (matches never-capture policy), 409 CAPTURE_DUPLICATE_URL_RECENT

13.2 Capture Status

GET /captures/{capture_id}/status

```
{ "success": true, "data": { "capture_id": "cap_44de", "status":
"completed", "memory_id": "mem_7f2a" } }
```

Status enum: queued, processing, completed, failed, cancelled.

13.3 Bulk Capture

POST /captures/bulk — accepts an array of up to 25 capture requests of mixed type; returns an array of individual capture_id/status pairs. Partial failures do not fail the whole batch (HTTP 207-style semantics represented as 200 with per-item status).

13.4 Retry / Cancel

- Retry only permitted when status = failed, else 409 CAPTURE_INVALID_STATE.
- Cancel only permitted when status $\in$ {queued, processing}, else 409 CAPTURE_INVALID_STATE.

14. Module: Upload

Method	Endpoint	Purpose
POST	/uploads/document	Generic text/markdown document
POST	/uploads/pdf	PDF upload
POST	/uploads/image	Image upload
GET	/uploads/{upload_id}/status	Get processing status
DELETE	/uploads/{upload_id}	Delete upload (and derived memory, if linked)

14.1 Upload PDF

POST /api/v1/uploads/pdf **Headers:** Content-Type: multipart/form-data, Idempotency-Key **Form fields:** file (binary, required), title (optional string)

202 Response:

```
{ "success": true, "data": { "upload_id": "upl_9c21", "status":  
"processing", "file_size_bytes": 2456123 } }
```

Errors: 413 UPLOAD_FILE_TOO_LARGE, 415 UPLOAD_INVALID_MIME_TYPE, 422
UPLOAD_VALIDATION_FAILED

15. Module: Search

Method	Endpoint	Purpose
GET	/search/semantic	Vector similarity search over Memory content
GET	/search/keyword	Traditional keyword/full-text search
GET	/search/hybrid	Combined semantic + keyword, ranked
GET	/search/suggestions	Autocomplete/query suggestions
GET	/search/recent	Recently run searches
GET	/search/saved	List saved searches
POST	/search/saved	Save a search query
DELETE	/search/saved/{saved_search_id}	Delete a saved search
GET	/search/history	Full search history (paginated)

15.1 Semantic Search

GET /api/v1/search/semantic?
q=how+does+pgvector+indexing+work&limit=10

200 Response:

```
{  
  "success": true,  
  "data": [  
    {  
      "memory_id": "mem_7f2a",  
      "title": "Understanding pgvector indexing",  
      "snippet": "...HNSW offers faster approximate search...",  
      "similarity_score": 0.87,  
      "source_type": "webpage"  
    }  
  ],  
  "pagination": { "cursor": null, "next_cursor": null, "has_more":  
false, "page_size": 10 }  
}
```

Notes: similarity_score is cosine similarity in [0,1], rounded to 2 decimals. Minimum score threshold (default 0.55) is applied server-side; not client-configurable in MVP.

15.2 Hybrid Search

GET /search/hybrid?q=...&weight=balanced — weight ∈ {semantic_heavy, balanced, keyword_heavy}, default balanced.

15.3 Saved Searches

POST /search/saved → { "query": "pgvector indexing", "filters": { "source_type": "webpage" } } → 201.

16. Module: AI

Method	Endpoint	Purpose
POST	/ai/ask	Ask a question over Memory (RAG chat)
GET	/ai/conversations	List past conversations
GET	/ai/conversations/{conversation_id}	Get conversation with message history
DELETE	/ai/conversations/{conversation_id}	Delete conversation
POST	/ai/conversations/{conversation_id}/messages	Continue a conversation
POST	/ai/summarize/{memory_id}	Generate summary with citation
POST	/ai/explain/{memory_id}	Explain a memory in simpler terms
POST	/ai/tags/generate	Auto-generate suggested tags for a memory
POST	/ai/timeline/generate	Generate a chronological timeline (P1 — see note)

Note: Streaming responses (Server-Sent Events / chunked transfer) are a **future (V1.x)** capability. MVP responses are synchronous, single-shot JSON. generate_timeline is listed per the module inventory but is **out of MVP scope** per the frozen PRD (Intelligence module MVP = Semantic Search + RAG Chat + Source Citations only); endpoint is documented here as a stub returning 501 NOT_IMPLEMENTED until V1.x.

16.1 Ask AI

POST /api/v1/ai/ask **Headers:** Idempotency-Key **Body:**

```
{
  "question": "What did I learn about vector indexing last month?",
  "conversation_id": null,
  "memory_scope": { "source_types": ["webpage", "pdf"], "date_from":
"2026-06-01" }
}
```

200 Response:

```

{
  "success": true,
  "data": {
    "conversation_id": "conv_5521",
    "answer": "You explored HNSW vs IVFFlat indexing for
pgvector...",
    "citations": [
      { "memory_id": "mem_7f2a", "title": "Understanding pgvector
indexing", "excerpt": "HNSW offers..." }
    ]
  }
}

```

Errors: 422 AI_QUESTION_TOO_LONG (>2000 chars), 503
 AI_PROVIDER_UNAVAILABLE, 429 SYSTEM_RATE_LIMITED

16.2 Summarize with Citations

POST /ai/summarize/{memory_id} → returns { "summary": "...",
 "citations": [...] }. Every generated summary sentence maps to at
 least one citation; summaries with no groundable citation are rejected
 server-side and return 422 AI_UNGROUNDED_SUMMARY.

16.3 Generate Tags

POST /ai/tags/generate — Body: { "memory_id": "mem_7f2a" } → returns
 up to 8 suggested tag strings; client decides whether to apply via the
 Tags module.

17. Module: Collections

Method	Endpoint	Purpose
POST	/collections	Create collection
PUT	/collections/{collection_id}	Update (name, description)
DELETE	/collections/{collection_id}	Delete collection
POST	/collections/{collection_id}/memories	Add memory to collection
DELETE	/collections/{collection_id}/memories/{memory_id}	Remove memory from collection
GET	/collections	List collections
GET	/collections/{collection_id}	Get single collection with memories

Validation: name 1–80 chars, unique per user (409
 COLLECTION_NAME_EXISTS on duplicate).

18. Module: Tags

Method	Endpoint	Purpose
--------	----------	---------

POST	/tags	Create tag
PUT	/tags/{tag_id}	Rename/update tag
DELETE	/tags/{tag_id}	Delete tag (detaches from all memories)
POST	/memories/{memory_id}/tags	Assign tag(s) to memory
DELETE	/memories/{memory_id}/tags/{tag_id}	Remove tag from memory
GET	/tags	List all tags (with usage counts)

Validation: Tag name 1-40 chars, lowercase-normalized server-side, unique per user. Duplicate → 409 TAG_ALREADY_EXISTS.

19. Module: Dashboard

Method	Endpoint	Purpose
GET	/dashboard/statistics	Aggregate stats (total memories, sources breakdown)
GET	/dashboard/recent-activity	Recent captures/edits/searches (unified feed)
GET	/dashboard/usage-summary	Weekly/monthly usage trend
GET	/dashboard/memory-count	Total memory count (with source breakdown)
GET	/dashboard/capture-statistics	Success/failure rates by capture type
GET	/dashboard/storage-usage	Bytes used vs. plan quota

All endpoints in this module are GET-only, read-optimized, and cached server-side for up to 5 minutes (Cache-Control: private, max-age=300 on response).

20. Module: Notifications

Method	Endpoint	Purpose
GET	/notifications	List notifications (paginated)
POST	/notifications/{notification_id}/read	Mark one as read
POST	/notifications/read-all	Mark all as read
DELETE	/notifications/{notification_id}	Delete notification

21. Module: Chrome Extension

These endpoints are consumed exclusively by the Manifest V3 extension client and share the same base URL and auth scheme as the web app, with a few extension-specific additions.

Method	Endpoint	Purpose
POST	/extension/authenticate	Exchange web session or credentials for extension-scoped tokens
POST	/extension/sync-session	Sync auth state between web app and extension
POST	/extension/capture/page	Capture the currently active tab's page
POST	/extension/capture/youtube	Capture the currently active YouTube tab
POST	/extension/capture/pdf	Capture a PDF open in the browser
POST	/extension/capture/article	Capture an article-type page
GET	/extension/capture/{capture_id}/status	Poll capture status (extension-scoped)
GET	/extension/health	Liveness check used by the extension background worker
GET	/extension/version	Minimum supported extension version + latest available version
POST	/extension/permissions/validate	Validate that granted browser permissions match required scopes

21.1 Authenticate Extension

POST /api/v1/extension/authenticate **Body:** { "web_session_token": "..."} (obtained via a short-lived handoff code from the web app) **or** { "email": "...", "password": "..."} for direct login inside the extension popup. **200 Response:** Same token shape as §10.2, with client_type: "extension" embedded in the JWT claims for scoping/analytics.

21.2 Health & Version

GET /extension/health → { "status": "ok" }, no auth required, used for background-worker connectivity checks. GET /extension/version → { "minimum_supported_version": "1.0.0", "latest_version": "1.2.0", "force_update": false }.

21.3 Permission Validation

POST /extension/permissions/validate **Body:** { "granted_permissions": ["activeTab", "storage", "scripting"] } **200 Response:** { "valid": true, "missing_permissions": [] } — used by the extension to prompt the user before attempting a capture that requires a permission it doesn't currently hold.

22. Module: Admin (Future)

All endpoints in this section are placeholders for a future internal-only Admin surface. Not part of MVP v0.1.
Require a separate admin role claim not issued to standard users.

Method	Endpoint	Purpose
GET	/admin/health	Deep health check (DB, Redis, Celery, MinIO)
GET	/admin/metrics	Prometheus-style metrics export
GET	/admin/logs	Query recent application logs
POST	/admin/users/{user_id}/moderate	Suspend/restore a user account
GET	/admin/system-status	Aggregate system status page data
GET	/admin/queue-status	Celery queue depth/backlog per queue

23. Webhooks (Future)

Placeholder — not implemented in MVP. Sentiora does not currently expose outbound webhooks. A future version may support user-configured webhooks for events such as `memory.created`, `capture.completed`, and `capture.failed`, delivered via signed HMAC payloads with retry/backoff. Specification to follow in a dedicated Webhooks API addendum once the Footprint/Shield roadmap stabilizes.

24. Security

- **HTTPS required** for all traffic; HTTP requests are rejected at the load balancer (no plaintext fallback).
- **JWT storage:** Web app prefers `HttpOnly`, `Secure`, `SameSite=Strict` cookies for the refresh token to mitigate XSS-based token theft; access tokens may be held in memory. Extension uses `chrome.storage.session/chrome.storage.local` as described in §2.

- **CSRF:** Because JWTs are also accepted via Authorization header (not cookie-only) for state-changing requests, classic CSRF is largely mitigated; where cookie-based refresh is used, a double-submit CSRF token is required on /auth/refresh-token and /auth/logout.
- **Input validation:** All request bodies are validated against Pydantic models server-side; unknown/extra fields are rejected (422) rather than silently ignored.
- **Output encoding:** All user-generated text (titles, notes, tags) is returned as raw JSON strings; client is responsible for safe rendering (no HTML is pre-rendered server-side).
- **Replay attack prevention:** Refresh token rotation (§2.5) plus rejection of reused, already-rotated tokens (invalidates entire token family). Idempotency-Key (§8) prevents duplicate side effects from retried writes.
- **Upload validation:** MIME-type allowlisting, size limits, and async malware scan before a file transitions to ready (§9).
- **Content Security Policy (extension context):** The Chrome Extension's injected content scripts operate under the host page's CSP; the extension does not inject inline scripts and communicates with the API only via the background service worker, never from content-script context directly.
- **Rate limiting:** As specified in §7, applied per-user/per-IP to blunt credential-stuffing and scraping.

25. Error Code Catalog

AUTH_*

Code	HTTP	Meaning	
AUTH_INVALID_CREDENTIALS	401	Email/password incorrect	S g e r f i v
AUTH_TOKEN_EXPIRED	401	Access token expired	A s r
AUTH_TOKEN_INVALID	401	Token malformed/tampered	F l
AUTH_FORBIDDEN	403	Valid token, insufficient permission	S p e r
AUTH_EMAIL_ALREADY_EXISTS	409	Registration email in use	F o r
AUTH_RESET_TOKEN_INVALID	400	Reset link expired/used	F t n li
AUTH_REFRESH_TOKEN_REUSED	401	Rotated refresh token replayed	F l (c

SEARCH_*

Code	HTTP	Meaning	Client Action
SEARCH_QUERY_TOO_SHORT	422	Query below min length (2 chars)	Prompt user to add more text
SEARCH_QUERY_TOO_LONG	422	Query exceeds 500 chars	Truncate/prompt user
SEARCH_INVALID_FILTER	422	Unknown filter key/value	Show validation message

CAPTURE_*

Code	HTTP	Meaning	Client Action
CAPTURE_URL_BLOCKED_SENSITIVE	422	URL matches never-capture policy	Inform user capture disabled for this page
CAPTURE_DUPLICATE_URL_RECENT	409	Same URL captured very recently	Offer view existing memo instead
CAPTURE_INVALID_STATE	409	Retry/cancel attempted in wrong state	Refresh capture status before retrying
CAPTURE_SOURCE_DISABLED	403	User has disabled this capture source in privacy settings	Prompt user to enable source settings

UPLOAD_*

Code	HTTP	Meaning
UPLOAD_FILE_TOO_LARGE	413	Exceeds size limit
UPLOAD_INVALID_MIME_TYPE	415	Disallowed file type
UPLOAD_VALIDATION_FAILED	422	Corrupt/empty/unreadable file
UPLOAD_MALWARE_DETECTED	422	Async scan flagged the file

AI_*

Code	HTTP	Meaning	Client Action
AI_QUESTION_TOO_LONG	422	Question exceeds 2000 chars	Prompt us to shorter
AI_UNGROUNDED_SUMMARY	422	No citable source for generated content	Show fallback message; not display summary
AI_PROVIDER_UNAVAILABLE	503	Upstream model provider down	Show retr option with backoff
AI_CONVERSATION_NOT_FOUND	404	Conversation ID invalid/not owned	Start a new conversation

SYSTEM_*

Code	HTTP	Meaning	Client Action
SYSTEM_RATE_LIMITED	429	Rate limit exceeded	Retry after backoff
SYSTEM_IDEMPOTENCY_KEY_REUSED	409	Same key, different payload	Generate new idempotency key
SYSTEM_MAINTENANCE	503	Planned maintenance window	Show maintenance banner
SYSTEM_INTERNAL_ERROR	500	Unhandled exception	Show generic error to users

26. OpenAPI / Documentation Strategy

- FastAPI's automatic OpenAPI 3.1 generation (`/api/v1/openapi.json`) is the source of truth for schema-level contract validation; this document is the human-readable companion and takes precedence on intent/behavior where the two diverge until reconciled.
- Interactive documentation is served at `/api/v1/docs` (Swagger UI) and `/api/v1/redoc` (ReDoc) in non-production environments; disabled or auth-gated in production.
- All Pydantic request/response models must include `Field(description=...)` annotations so generated OpenAPI docs stay meaningful without manual duplication.
- Contract tests (e.g. Schemathesis or Dredd) should run against the

generated OpenAPI spec in CI to catch drift between this document and actual implementation.

27. API Naming Rules

- Plural nouns for collections: /memories, /collections, /tags.
- Lowercase, kebab-case for multi-word resources: /saved-searches, /notification-settings.
- No verbs in URLs except where REST conventions require an action on a sub-resource that isn't naturally CRUD (e.g. /memories/{id}/archive, /memories/{id}/restore, /captures/{id}/retry) — these are treated as sub-resource actions (POST to a verb-like sub-path), not general precedent for verb-based routing.
- Path parameters use {resource_singular}_id (e.g. {memory_id}, {collection_id}).
- Query parameters use snake_case.
- JSON body fields use snake_case throughout (matches Python/FastAPI/Pydantic conventions and avoids case-mapping bugs between the Chrome Extension's JS and the backend).

End of Sentiora API Specification v1.0.